RITUAL – HABIT TRACKER

A Python-Based Desktop Application for Daily Habit Management

PIP MINI-PROJECT REPORT

By

SHAUN M BABU (AJC23CS186)

S6 CSE C

Roll No: 46

ABSTRACT

Ritual is a desktop habit-tracking application developed entirely in Python using the Tkinter graphical user interface library. The application enables users to define, monitor, and analyse personal habits with the goal of building consistent, productive daily routines. Key features include habit creation with customisable frequency, category, colour, and reminder time; real-time streak tracking; a built-in Pomodoro focus timer; detailed analytics with heatmaps and bar charts; a daily journal with mood logging; CSV and JSON data export; and a dedicated settings page. The project demonstrates the practical application of Python GUI programming, data persistence using JSON, multi-threading for background reminders, and object-oriented software design.

PROBLEM DESCRIPTION

Maintaining consistent habits is a widely recognised challenge in personal productivity. Most people struggle to track their daily routines without a reliable system, often relying on memory or paper-based methods that are difficult to sustain long-term. Commercial habit-tracking applications typically require internet access, subscriptions, or mobile devices, creating accessibility barriers for many users.

This project addresses the following specific problems:

- Lack of a simple, offline, cross-platform habit tracking tool built purely in Python.
- No easy way to visualise habit consistency over time using only the standard library.
- Difficulty maintaining streaks without automated reminders or visual feedback.
- Absence of journaling and reflection features integrated directly into a habit tool.
- No mechanism to pause (archive) a habit temporarily without losing historical data.
- Inability to log missed past days retroactively, causing artificially broken streaks.

The solution — Ritual — is a fully offline, single-file Python application that requires no external dependencies beyond the Python standard library, making it lightweight, portable, and accessible on any system with Python installed.

LIBRARIES & TOOLS USED

The following Python standard-library modules and tools were used in the project:

Library / Tool	Module / Version	Purpose
Tkinter	tkinter (stdlib)	Primary GUI framework — windows, widgets, layout
ttk	tkinter.ttk	Themed widgets: Combobox, Progressbar, Scrollbar
JSON	json (stdlib)	Persistent data storage — read/write habits.json
OS	os (stdlib)	File path resolution, data file location
CSV	csv (stdlib)	Exporting habit data to spreadsheet-compatible CSV
Threading	threading (stdlib)	Background thread for reminder time-checking
Time	time (stdlib)	Sleep intervals in the reminder polling loop
Random	random (stdlib)	Daily challenge — random incomplete habit picker
Datetime	datetime (stdlib)	Date arithmetic: streaks, completion windows
filedialog	tkinter.filedialog	Native OS file open/save dialogs for import/export
messagebox	tkinter.messagebox	User alerts, confirmation dialogs, reminders
Python 3.10+	Runtime	Language runtime — f-strings, match, type hints
Git	CLI Tool	Version control and GitHub repository management

SYSTEM DESIGN / WORKFLOW

4.1 Architecture Overview

The application follows an object-oriented, single-file MVC-inspired architecture. All application state is managed by the central App class, which owns the data dictionary and passes itself as a reference to each page. Pages are Tkinter Frame subclasses that are layered on top of each other and raised to the front when selected.

4.2 Data Model

All data is stored in a single habits.json file with the following top-level structure:

Key	Type	Description
habits	Object (dict)	Maps habit name to its configuration and completions list
journal	Object (dict)	Maps ISO date string to journal entry (text, mood, tags)
settings	Object (dict)	User preferences: name, grace day, sound, archived view

Each habit object contains the following fields:

- name — unique string identifier
- type — 'build' or 'break'
- freq — 'daily', 'weekly', or 'monthly'
- category — one of: Health, Fitness, Learning, Mindfulness, Productivity, etc.
- color — hex colour accent string (e.g. #7b61ff)
- emoji — optional single emoji character
- goal — motivational text string
- remind — HH:MM time string for reminder notification
- target — integer, number of completions per period
- completions — list of ISO date strings when the habit was completed
- archived — boolean flag to hide without deleting

4.3 Page / Module Structure

Class	Page	Responsibility
App	Root window	Sidebar, navigation, undo stack, reminder thread, export
DashboardPage	Dashboard	Today's habits, progress bar, challenge, search, filter
HabitsPage	My Habits	Full CRUD, archive/restore, 30-day dot map
AnalyticsPage	Analytics	Heatmap, per-habit bars, leaderboard, best-day, weekly chart
JournalPage	Journal	Date-indexed entries with mood, tags, and habit notes
SettingsPage	Settings	User name, grace day, sound, export/import, statistics
HabitDialog	Modal dialog	Create / edit habit form with all properties
BackfillDialog	Modal dialog	Log completions for up to 3 past missed days
FocusTimer	Modal window	Pomodoro-style countdown timer with preset durations

4.4 Workflow

1. Application launches and loads habits.json (or creates default structure if absent).
2. User navigates between pages using the left sidebar.
3. On Dashboard: habits are displayed sorted by completion status. User taps Mark Done to log today.
4. Background thread wakes every 30 seconds and checks reminder times; fires a popup if due.
5. Analytics page computes streak, completion rate, and day-of-week statistics from raw completion lists.
6. Journal entries are saved per date with mood, free text, optional tags, and per-habit notes.
7. All writes call save() which serialises the in-memory dict back to habits.json immediately.

IMPLEMENTATION DETAILS

5.1 Streak Algorithm

The streak() function iterates backward through sorted completion dates. Starting from today, it decrements the check date by one day for each matching completion. If the optional grace flag is enabled (Settings), one missed day per run is skipped without breaking the count. This handles the common case of a user missing a single day while still maintaining a meaningful streak.

5.2 Reminder System

A daemon thread is started on application launch and polls every 30 seconds. It compares the current HH:MM time against each habit's reminder field. If they match and the habit has not yet been completed today, it schedules a messagebox popup on the main thread using Tkinter's after(0, ...) callback mechanism, which is thread-safe. Each habit is only reminded once per day using a session-level set.

5.3 Focus Timer

The FocusTimer window uses Tkinter's after(1000, ...) recursive scheduling to decrement a countdown counter every second without blocking the main GUI thread. Four preset durations (5, 15, 25, 50 minutes) are provided. A Canvas widget renders a simple horizontal progress bar that fills as time elapses. On completion, the system bell is rung.

5.4 Undo System

Every call to _toggle() pushes a (habit_name, date_str, was_added) tuple onto a session-level undo stack stored in the App instance. The sidebar Undo button pops the last entry and reverses the operation — removing a completion if it was added, or re-adding it if it was removed.

5.5 Analytics Visualisation

All charts are built entirely from Tkinter Frame widgets — no external charting library is used. The 30-day heatmap maps each day's combined completion fraction to one of four purple shades. Bar charts are rendered as fixed-height Frame widgets with calculated widths. The best-day-of-week chart computes a per-weekday completion rate over the last 90 days to normalise for the number of times each weekday has occurred.

5.6 Data Export

CSV export uses Python's csv module to write one row per habit with name, category, type, frequency, streak, 7-day rate, 30-day rate, total completions, and goal. JSON backup serialises the entire data dictionary. Both use filedialog.asksaveasfilename for native OS save-file dialogs. Import reads a JSON file and replaces the in-memory data after a confirmation prompt, then re-saves and refreshes the current page.

RESULTS & OUTPUT

The application was tested on Windows 11 with Python 3.11. All features functioned as expected. The following outputs were verified during testing:

6.1 Dashboard

- Stat cards correctly showed completed/total counts, best streak, and 7-day average.
- Progress bar filled proportionally as habits were marked done.
- Daily challenge banner surfaced a random incomplete habit each session.
- Search and category filter correctly narrowed the habit list in real time.
- Urgency indicator (red border) appeared for habits past their reminder time.

6.2 Habit Management

- New habit dialog accepted all fields and saved correctly to habits.json.
- Edit preserved existing completion history while updating all other fields.
- Archive hid habits from Dashboard and Analytics without deleting data.
- 30-day dot map rendered accurately, matching the raw completions list.

6.3 Analytics

- Heatmap shading intensity correctly reflected combined daily completion volume.
- Streak leaderboard sorted habits by current streak in descending order.
- Best-day-of-week chart highlighted the weekday with highest historical rate.
- Weekly bar chart covered 8 weeks of history with accurate percentage labels.

6.4 Focus Timer

- All preset durations (5, 15, 25, 50 min) set the countdown correctly.
- Pause and resume functioned without drift. Bell rang on completion.
- Progress bar filled smoothly from left to right as time elapsed.

6.5 Journal

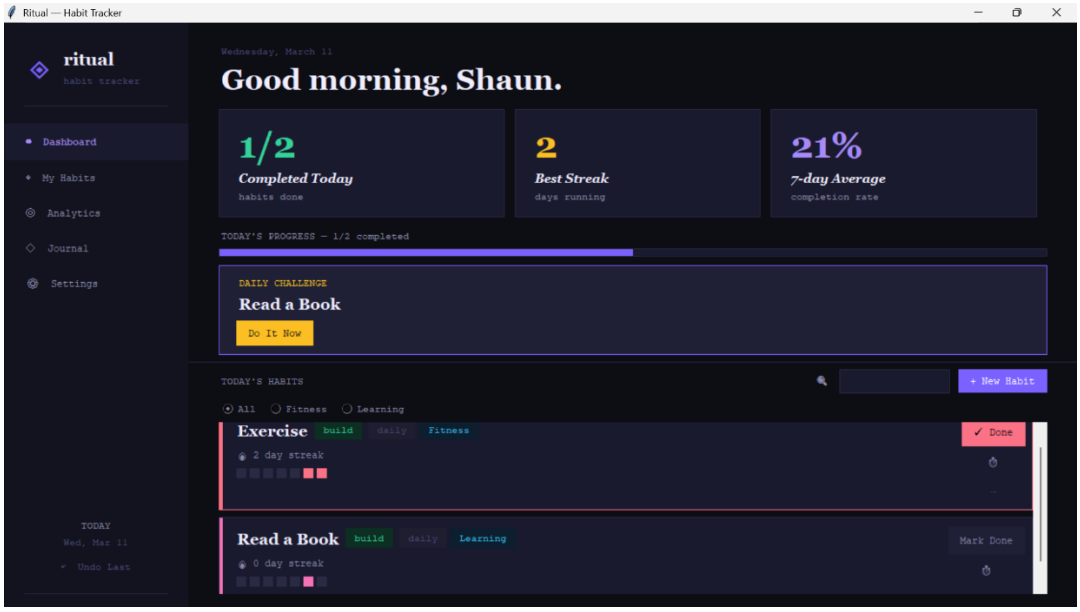
- Entries saved per date with mood emoji, tags, reflection text, and habit notes.
- Entry list sidebar previewed the first 38 characters of each entry.

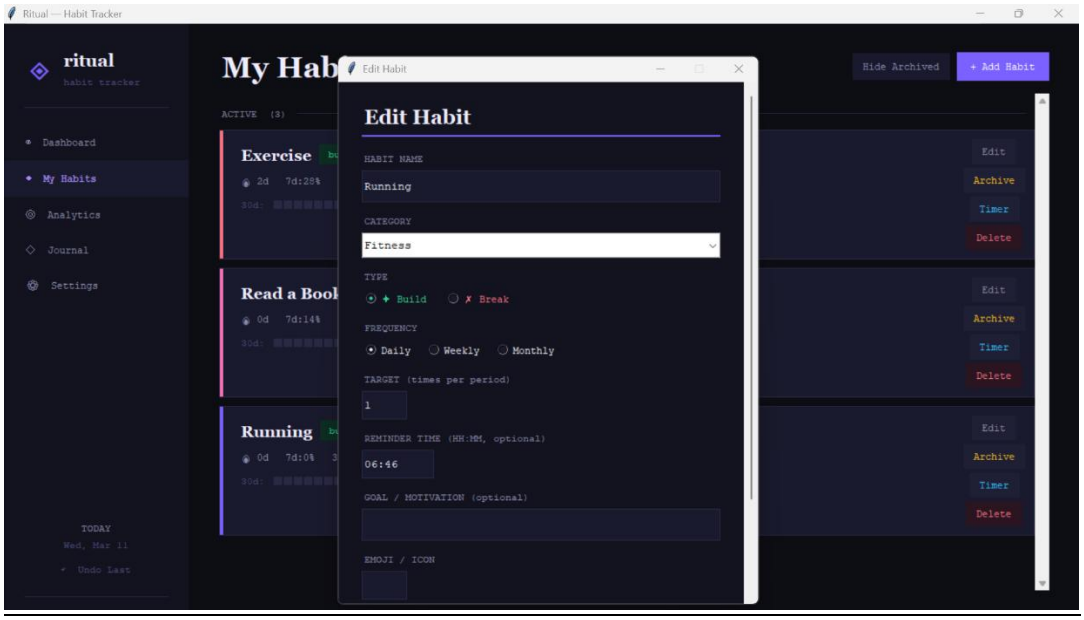
- Mood icon displayed next to date in the sidebar list.

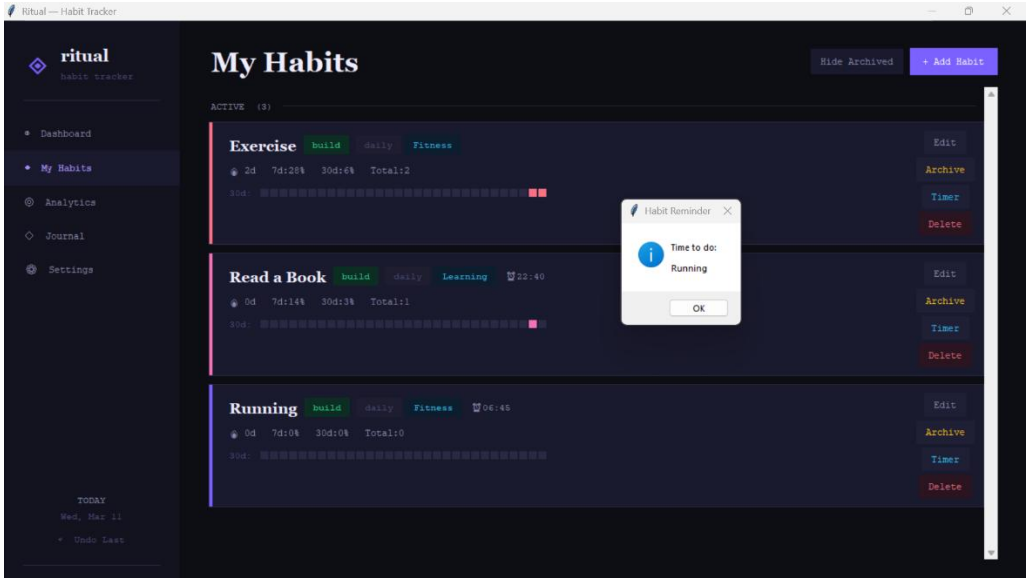
6.6 Settings & Export

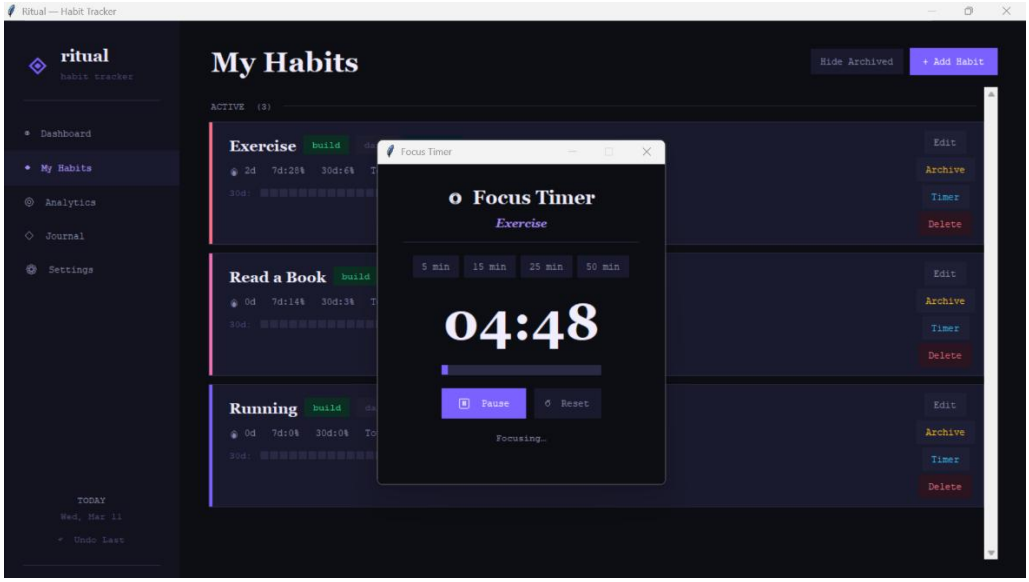
- Username change updated the Dashboard greeting immediately after save.
- Streak grace day toggle correctly skipped one missed day in streak calculations.
- CSV export produced a correctly formatted file openable in Microsoft Excel.
- JSON backup and restore cycle preserved all data without loss.

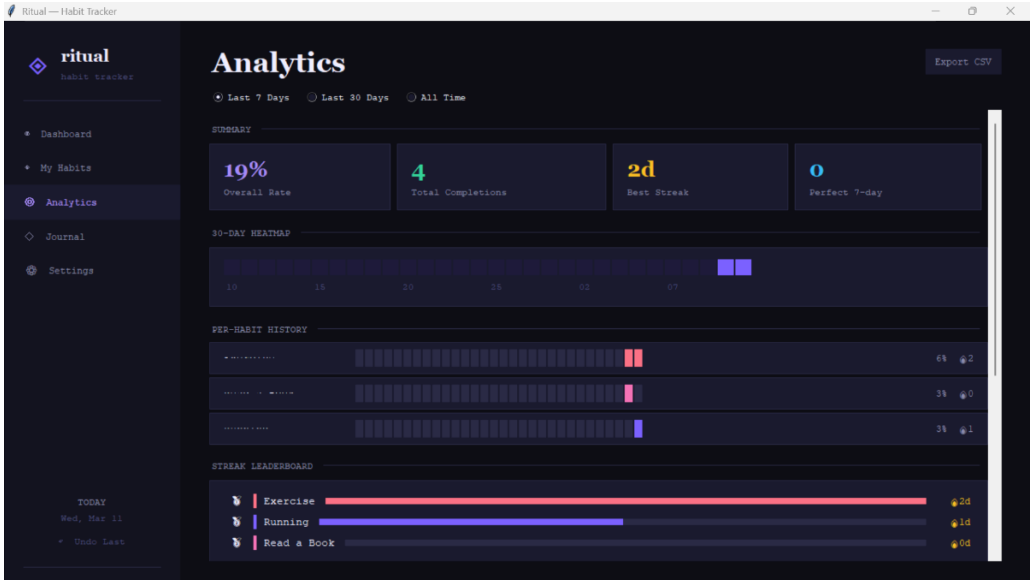
SCREENSHOTS

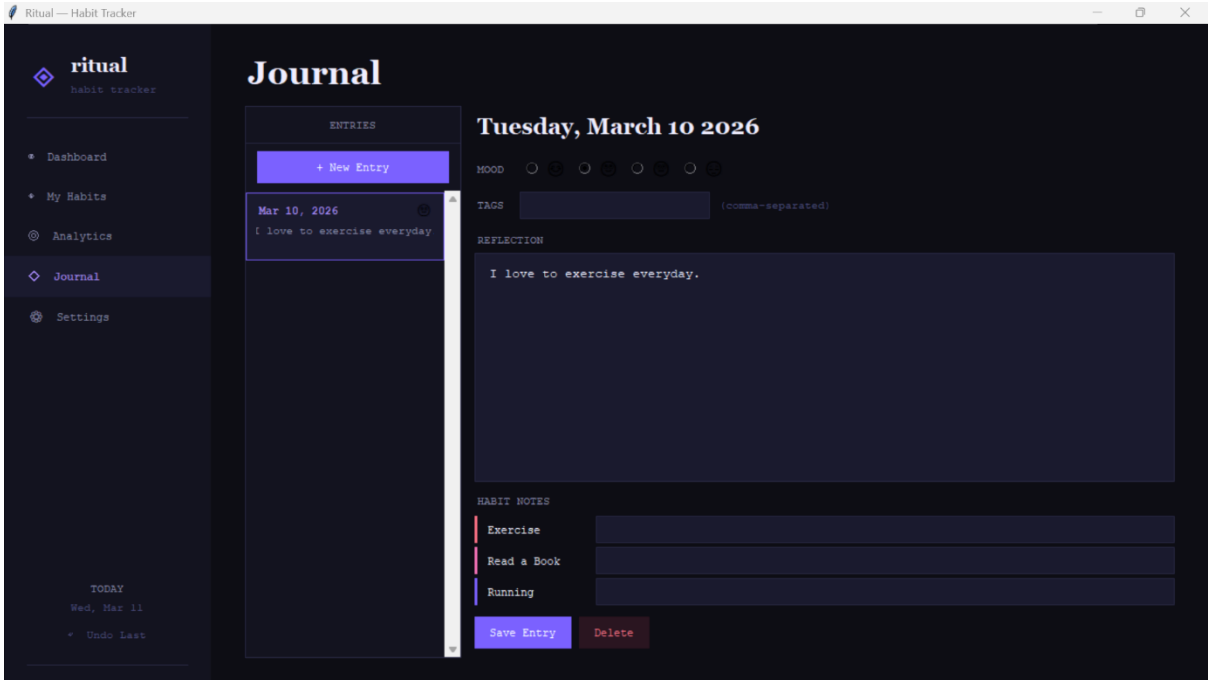


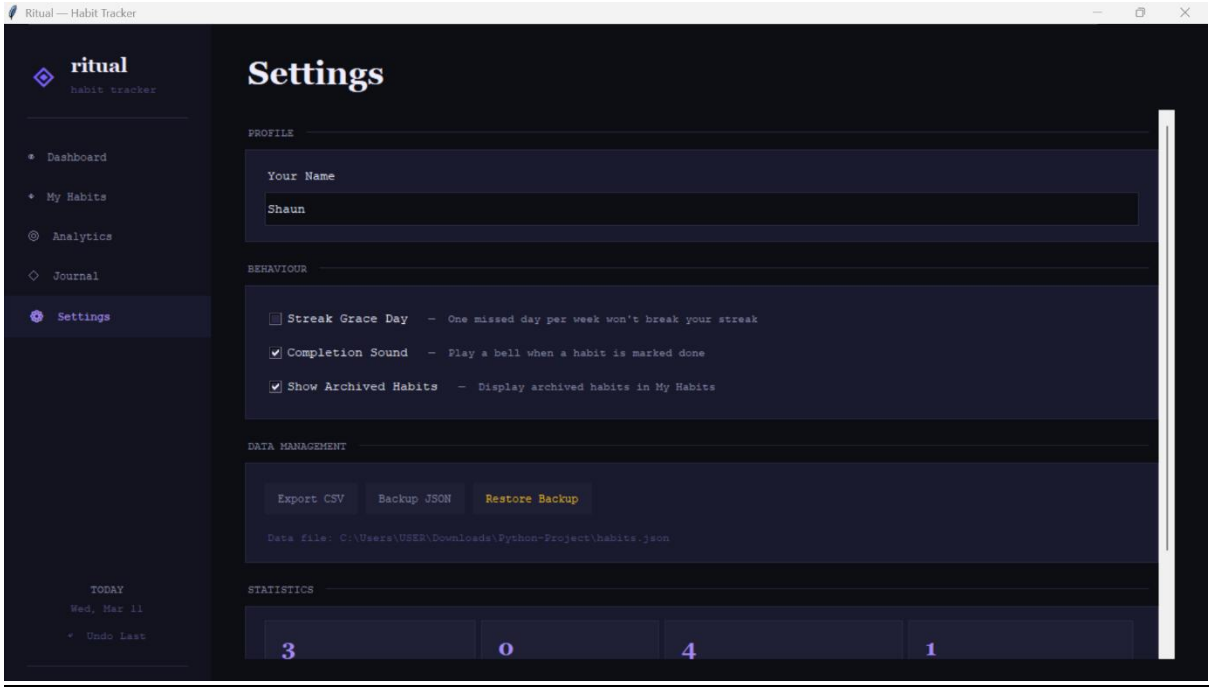



Ritual — Habit Tracker
ritual
habit tracker
Dashboard
My Habits
Analytics
Journal
Settings
TODAY
Wed, Mar 11
Undo Last
My Habits
Hide Archived
+ Add Habit
ACTIVE (3)
Exercise
build
daily
Fitness
2d 7d:28% 30d:6% Total:2
Edit
Archive
Timer
Delete
Habit Reminder
Time to do:
Running
OK
Read a Book
build
daily
Learning
22:40
0d 7d:14% 30d:3% Total:1
Running
build
daily
Fitness
06:45
0d 7d:0% 30d:0% Total:0


Ritual — Habit Tracker
ritual
habit tracker
Dashboard
My Habits
Analytics
Journal
Settings
My Habits
Hide Archived
+ Add Habit
ACTIVE (3)
Exercise
Focus Timer
Focus Timer
Exercise
5 min
15 min
25 min
50 min
04:48
Pause
Reset
Focusing...
Read a Book
Running


Ritual — Habit Tracker
ritual
habit tracker
Dashboard
My Habits
Analytics
Journal
Settings
Analytics
Export CSV
Last 7 Days
Last 30 Days
All Time
SUMMARY
19%
Overall Rate
4
Total Completions
2d
Best Streak
0
Perfect 7-day
30-DAY HEATMAP
PER-HABIT HISTORY
STREAK LEADERBOARD
Exercise
Running
Read a Book


Ritual — Habit Tracker
ritual
habit tracker
Dashboard
My Habits
Analytics
Journal
Settings
TODAY
Wed, Mar 11
Undo Last
Journal
ENTRIES
+ New Entry
Mar 10, 2026
I love to exercise everyday
Tuesday, March 10 2026
MOOD
TAGS
(comma-separated)
REFLECTION
I love to exercise everyday.
HABIT NOTES
Exercise
Read a Book
Running
Save Entry
Delete


Ritual — Habit Tracker
ritual
habit tracker
Dashboard
My Habits
Analytics
Journal
Settings
TODAY
Wed, Mar 11
Undo Last
Settings
PROFILE
Your Name
Shaun
BEHAVIOUR
Streak Grace Day — One missed day per week won't break your streak
Completion Sound — Play a bell when a habit is marked done
Show Archived Habits — Display archived habits in My Habits
DATA MANAGEMENT
Export CSV
Backup JSON
Restore Backup
Data file: C:\Users\USER\Downloads\Python-Project\habits.json
STATISTICS
3
0
4
1

CONCLUSION & SCOPE FOR EXTENSION

7.1 Conclusion

The Ritual Habit Tracker successfully demonstrates the practical capabilities of Python and its standard library for building fully functional, visually polished desktop applications. The project covers a wide range of programming concepts including object-oriented design, event-driven GUI programming, file I/O, multi-threading, data visualisation, and algorithmic thinking.

All defined requirements were implemented and verified. The application provides genuine utility as a daily productivity tool, with features comparable to commercial habit trackers while remaining entirely free, offline, and open source. The codebase is well-structured and maintainable, providing a solid foundation for future extension.

7.2 Scope for Mini Project Extension

The following features could be added in a future version of the project:

Extension	Description
Cloud Sync	Sync habits.json to Google Drive or Dropbox via their Python APIs
Mobile Companion	A Kivy or BeeWare port to run the tracker on Android or iOS
Notifications (OS)	Replace messagebox reminders with native OS notifications (plyer)
SQLite Backend	Migrate from JSON to SQLite for faster queries on large datasets
Habit Templates	Pre-built habit packs (Morning Routine, Fitness Plan, etc.)
Social / Sharing	Share streak milestones or weekly reports as images
AI Suggestions	Use an LLM API to suggest new habits based on journal reflections
Dark/Light Theme	Theme toggle with user-selectable colour palettes
Weekly Review Page	Automated weekly summary with insights and recommendations
Habit Dependencies	Chain habits so Habit B only unlocks after Habit A is done

REFERENCES

1. Python Software Foundation. Python 3.11 Documentation. https://docs.python.org/3/
2. Python Software Foundation. Tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
3. Duhigg, C. (2012). The Power of Habit: Why We Do What We Do in Life and Business. Random House.
4. Clear, J. (2018). Atomic Habits: An Easy & Proven Way to Build Good Habits & Break Bad Ones. Avery.
5. Effbot.org. An Introduction to Tkinter. http://effbot.org/tkinterbook/
6. GitHub Docs. Adding a file to a repository. https://docs.github.com/en/repositories
7. Real Python. Python GUI Programming With Tkinter. https://realpython.com/python-gui-tkinter/
8. Cirillo, F. (2006). The Pomodoro Technique. FC Garage, Berlin.